

3) WATET JUG PROBLEM

Aim

To develop a Python program to solve the Water Jug Problem using the Breadth-First Search (BFS) algorithm..

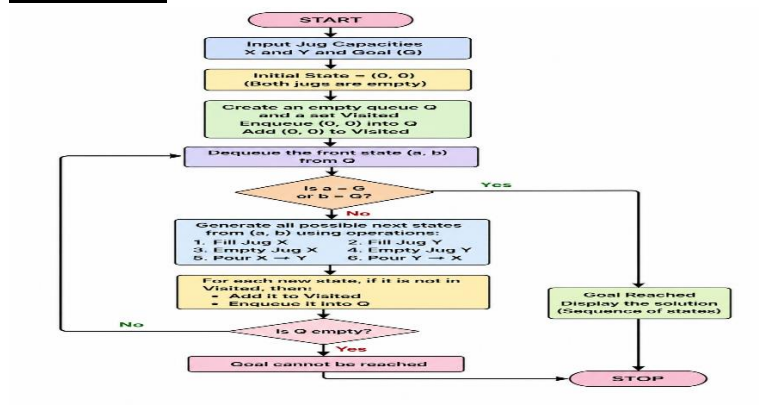
Objectives

- To understand the Water Jug Problem in Artificial Intelligence.
- To apply the Breadth-First Search (BFS) algorithm to find the solution.
- To obtain the required amount of water using two jugs.
- To identify the shortest sequence of operations to reach the goal state.

Algorithm

1. Start the program.
2. Define the capacities of the two water jugs.
3. Set the initial state as both jugs empty.
4. Generate all possible states by:
5. Filling a jug.
6. Emptying a jug.
7. Pouring water from one jug to another.
8. Mark visited states to avoid repetition.
9. Continue searching until the goal amount is obtained.
10. Display the sequence of states.
11. Stop the program.

Flow Chart



Python Program

```
from collections import deque
```

```
def water_jug(x, y, goal):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        a, b = queue.popleft()

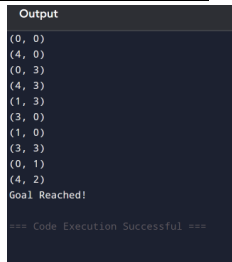
        if (a, b) in visited:
            continue

        visited.add((a, b))
        print((a, b))

        if a == goal or b == goal:
            print("Goal Reached!")
            return

        next_states = [
            (x, b),      # Fill Jug X
            (a, y),      # Fill Jug Y
            (0, b),      # Empty Jug X
            (a, 0),      # Empty Jug Y
            (max(0, a-(y-b)), min(y, a+b)), # X → Y
            (min(x, a+b), max(0, b-(x-a))) # Y → X
        ]
        for state in next_states:
            if state not in visited:
                queue.append(state)
water_jug(4, 3, 2)
```

Sample Output



```
Output
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
Goal Reached!

=== Code Execution Successful ===
```

Result

Thus, the Python program to solve the Water Jug Problem using the Breadth-First Search (BFS) algorithm was implemented successfully, and the required amount of water was obtained.

Conclusion

The Water Jug Problem demonstrates the application of the Breadth-First Search (BFS) algorithm in Artificial Intelligence. The algorithm systematically explores all possible states and successfully finds the shortest sequence of operations to reach the desired amount of water.

4) CRIPT-ARITHMETIC PROGRAM

Aim

To develop a Python program to solve the Cryptarithmic Problem using Artificial Intelligence techniques.

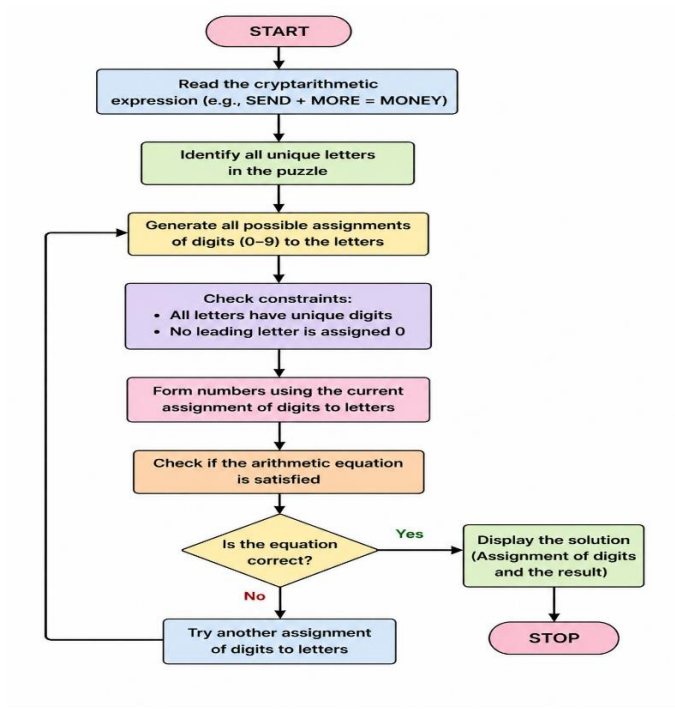
Objectives

- To understand the Cryptarithmic Problem in Artificial Intelligence.
- To assign unique digits to letters without repetition.
- To verify the arithmetic equation using the assigned digits.
- To obtain the correct solution satisfying all constraints.

Algorithm

1. Start the program.
2. Read the cryptarithmic expression.
3. Identify all unique letters in the puzzle.
4. Assign a unique digit (0–9) to each letter.
5. Ensure that no leading letter is assigned zero.
6. Check whether the arithmetic equation is satisfied.
7. If the equation is correct, display the solution.
8. Otherwise, try another digit assignment.
9. Stop when a valid solution is found.

Flow Chart



Python Program

```
from itertools import permutations
```

```
letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
```

```
for p in permutations(range(10), 8):
```

```
    S, E, N, D, M, O, R, Y = p
```

```
    if S == 0 or M == 0:
```

```
        continue
```

```
    SEND = 1000*S + 100*E + 10*N + D
```

```
    MORE = 1000*M + 100*O + 10*R + E
```

```
    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y
```

```
    if SEND + MORE == MONEY:
```

```
        print("SEND =", SEND)
```

```
        print("MORE =", MORE)
```

```
        print("MONEY =", MONEY)
```

break

Sample Output

```
Output
SEND = 9567
MORE = 1085
MONEY = 10652

=== Code Execution Successful ===
```

Result

Thus, the Python program to solve the Cryptarithmic Problem was implemented successfully, and the correct digit assignment satisfying the given equation was obtained.

Conclusion

The Cryptarithmic Problem demonstrates the use of constraint satisfaction in Artificial Intelligence. By assigning unique digits to letters and checking the arithmetic constraints, the program successfully finds the correct solution.